

Cours 12 – Pointillisme et distance

Fichiers : ofApp12.h + ofApp12.cpp , image data/pandaroux.jpg **Avant** : cours 11 (double boucle sur une image), cours 10 (lire un pixel).

L'image n'est plus affichée : elle est **redessinée** avec des cercles, un tous les 20 pixels, chacun de la couleur du pixel qu'il remplace. Et près de la souris, les cercles rétrécissent. Pour ça il faut savoir mesurer une distance.



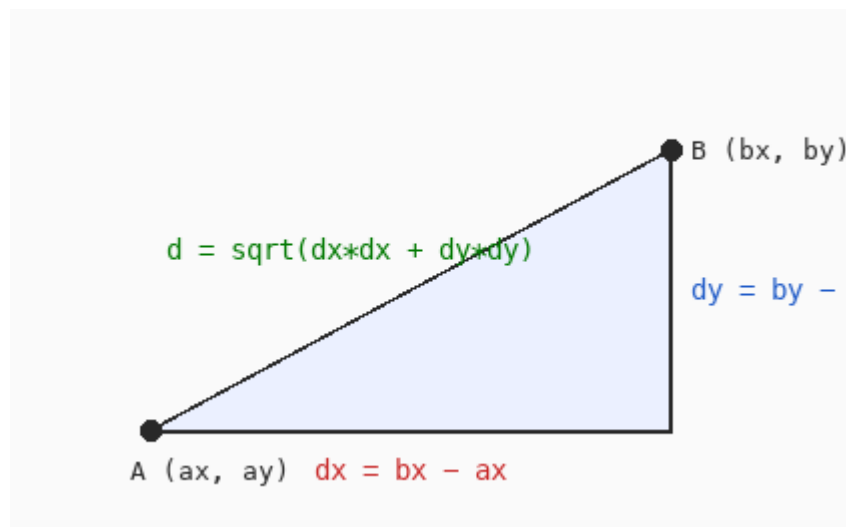
■ 1. Une grille : la double boucle avec un pas

```
for (int li = 0; li < windowHeight; li += tailleCercle) {  
    for (int co = 0; co < windowWidth; co += tailleCercle) {  
        ofColor c = img.getColor(co, li);  
        ofSetColor(c);  
        ofDrawCircle(co, li, rayon);  
    }  
}
```

La double boucle du cours 11, mais les compteurs avancent de 20 en 20 (`+= tailleCercle`, cours 03). Au lieu de visiter les 400 000 pixels, on en visite un sur 400, et on dessine à chaque fois un cercle de rayon 10 : les cercles se touchent, l'image est reconstituée en gros points.

L'image n'est jamais affichée avec `img.draw`. On lit ses pixels comme une source de données et on dessine autre chose à partir d'eux. C'est la remarque de la fin du cours 10 en action.

■ 2. La distance : Pythagore



```
float ofApp::distance(float ax, float ay, float bx, float by) {  
    float dx = bx - ax;  
    float dy = by - ay;  
    return sqrt(dx * dx + dy * dy);  
}
```

Deux points, un écart horizontal `dx`, un écart vertical `dy`. La distance est l'hypoténuse : racine carrée de $dx^2 + dy^2$. `sqrt` est la racine carrée. La fonction renvoie un `float` avec `return`, comme `filtre` renvoyait une `ofColor` au cours 11.

`openFrameworks` fournit `ofDist(ax, ay, bx, by)` qui fait exactement ceci. On l'a écrite pour la voir une fois ; ensuite tu peux utiliser `ofDist`.

■ 3. Une proportion

```
float d = distance(x, y, co, li);  
float rayon = tailleCercle / 2.0f;  
if (d <= 150) {  
    float proportion = d / 150.0f;  
    rayon = rayon * proportion;  
}
```

Pour chaque cercle on mesure sa distance à la souris. S'il est à moins de 150 pixels, on calcule `d / 150` : un nombre entre 0 (sur la souris) et 1 (à 150 pixels). Le rayon est multiplié par cette proportion : nul sous la souris, normal à 150 pixels, et tout ce qu'il y a entre.

Ce motif, **ramener une grandeur entre 0 et 1 puis multiplier**, est le moyen le plus simple de faire dépendre une chose d'une autre en douceur. Au lieu d'un `if` brutal « près / loin », on a un dégradé.

■ 4. Un effet local

L'ensemble donne un effet **local** : le même dessin partout, sauf dans un disque autour de la souris où il est modifié progressivement. Cette idée est indépendante de ce qu'on dessine. Elle se combine avec n'importe quel filtre du cours 11 : appliquer le négatif seulement aux pixels proches de la souris, c'est un filtre plus un **masque** défini par une distance.

■ 5. Variante : le temps dans la formule

En commentaire dans le code, la variante d'un autre sketch : des carrés dont la taille dépend de la distance à un anneau qui s'élargit avec le temps.

```
float t = ofGetElapsedTimef() * 60;
float prop = d / (200.0f + t);
ofDrawRectangle(co, li, tailleCercle * prop, tailleCercle * prop);
```

Même mécanique, un `t` en plus dans la proportion : l'effet grandit. Combiner la distance et le temps du cours 08 donne des animations riches pour trois lignes.

■ Exercices

1. Remplace les cercles par des carrés, puis par des rectangles dont la hauteur dépend de la luminosité du pixel.
2. Inverse l'effet : les cercles grossissent près de la souris.
3. **Filtre local** : reprends un filtre du cours 11 (négatif, gris, seuil) et applique-le seulement aux cercles à moins de 150 pixels de la souris.
4. Change `tailleCercle` : 5, 10, 40. À partir de quand ne reconnaît-on plus l'image ?
5. **Distance entre couleurs** : choisis cinq couleurs. Pour chaque cercle, calcule la distance dans le cube RGB (cours 09) entre le pixel et chacune des cinq, avec `sqrt(dr*dr + dg*dg + db*db)`, et dessine avec la plus proche. L'image en cinq couleurs.

■ Le code complet, pas à pas

Le programme entier, dans l'ordre des fichiers. Les blocs ci-dessous mis bout à bout donnent exactement `ofApp12.cpp`.

Le fichier `ofApp12.h`

La table des matières du programme : les blocs qui existent et les variables partagées entre eux, avec leurs valeurs de départ.

```
#pragma once
#include "ofMain.h"

// 12 - Pointillisme : boucles imbriquées, distance, image
// Sketch d'origine : Cours 2022-2023/rendu04
// Variante intégrée : Cours 2024-2025/sketch_03_image (carrés, anneau qui s'étend avec le temps)
// Notions : double boucle for (celle de 11, avec un pas de 20), distance euclidienne (Pythagore),
//          proportion, lecture de pixel dans une grille, effet local autour de la souris
// Ressource : bin/data/pandaroux.jpg (774 x 516)

class ofApp : public ofBaseApp {
public:
    void setup();
    void draw();

    float distance(float ax, float ay, float bx, float by);

    ofImage img;
    int windowHeight = 774;
    int windowHeight = 516;
    int tailleCercle = 20;
};
```

En tête du fichier `ofApp12.cpp`

L'inclusion du `.h`, qui rend visibles les variables partagées et les blocs déclarés.

```
#include "ofApp12.h"
```

Étape 1 — `setup()`

Exécuté une fois au lancement : la fenêtre, les chargements, les valeurs de départ.

```
void ofApp::setup() {
    ofSetWindowShape(windowWidth, windowHeight);
    if (!img.load("pandaroux.jpg")) {
        ofLogError() << "pandaroux.jpg introuvable dans bin/data";
    }
}
```

Étape 2 — `distance()`

Distance entre deux points : théorème de Pythagore.

```
// Distance entre deux points : théorème de Pythagore.  
// (openFrameworks fournit ofDist(ax, ay, bx, by) qui fait exactement ceci.)  
float ofApp::distance(float ax, float ay, float bx, float by) {  
    float dx = bx - ax;  
    float dy = by - ay;  
    return sqrt(dx * dx + dy * dy);  
}
```

Étape 3 — `draw()`

Exécuté à chaque frame, après `update()` : uniquement du dessin.

```

void ofApp::draw() {
    ofBackground(0);

    // Le sketch Processing affichait l'image puis lisait ses pixels à l'écran.
    // Ici on lit directement les pixels de l'ofImage, donc pas besoin de l'afficher.

    float x = mouseX;
    float y = mouseY;

    ofFill();
    // Pour une ligne li qui va de 0 à la hauteur, de 20 en 20 :
    for (int li = 0; li < windowHeight; li += tailleCercle) {
        // Pour une colonne co qui va de 0 à la largeur, de 20 en 20 :
        for (int co = 0; co < windowWidth; co += tailleCercle) {

            float d = distance(x, y, co, li);

            // Près de la souris, les cercles rétrécissent proportionnellement à la
            float rayon = tailleCercle / 2.0f;
            if (d <= 150) {
                float proportion = d / 150.0f;
                rayon = rayon * proportion;
            }

            // Couleur du pixel à cet endroit de l'image
            ofColor c = img.getColor(co, li);
            ofSetColor(c);
            ofDrawCircle(co, li, rayon);
        }
    }

    // ---- Variante (sketch_03_image, 2024-2025) ----
    // Remplacer le cercle par un carré dont la taille dépend de la distance
    // à un anneau qui s'élargit avec le temps :
    //
    // float t = ofGetElapsedTimef() * 60; // le sketch comptait en frames
    // float prop = d / (200.0f + t);
    // ofDrawRectangle(co, li, tailleCercle * prop, tailleCercle * prop);
    //
    // Exercice : filtre local. Reprendre un filtre de 11 (négatif, gris, seuil...) et ne l'appl:
    // qu'aux cercles situés à moins de 150 px de la souris : un filtre se combine avec un masque
    // Variante : ne plus dessiner des cercles mais la couleur la plus proche parmi 5 couleurs
    // choisies, en comparant par distance dans le cube RGB (Pythagore en 3D, cours 09).
}

```

■ Ce qu'il faut retenir

- Une double boucle avec un pas (`+= n`) parcourt une grille ; un pixel sur `n × n` est lu.

• Distance entre deux points : `sqrt(dx*dx + dy*dy)`, ou `ofDist(ax, ay, bx, by)`.

- Proportion $d / d_{\max}$ entre 0 et 1, puis multiplier : faire dépendre une grandeur d'une autre en douceur.
- Une image peut servir de source de données sans être affichée.
- Un effet local = un filtre plus un masque défini par une distance.